

# BLUEPRINT V1.1

AUTONOMOUS SWING TRADING BOT

FINAL / ARCHITECTURE FROZEN

Sandbox / Paper / Demo — Educational & Experimental

|                        |                             |
|------------------------|-----------------------------|
| Versão                 | V1.1                        |
| Status                 | FINAL / ARCHITECTURE FROZEN |
| Capital de referência  | R\$ 100,00                  |
| Fases de implementação | 16                          |
| Live Broker            | NÃO IMPLEMENTADO            |
| Data                   | 2026-08-15                  |

# SUMÁRIO EXECUTIVO

Este documento é o baseline arquitetural e de implementação da V1.1 do Autonomous Swing Trading Bot.

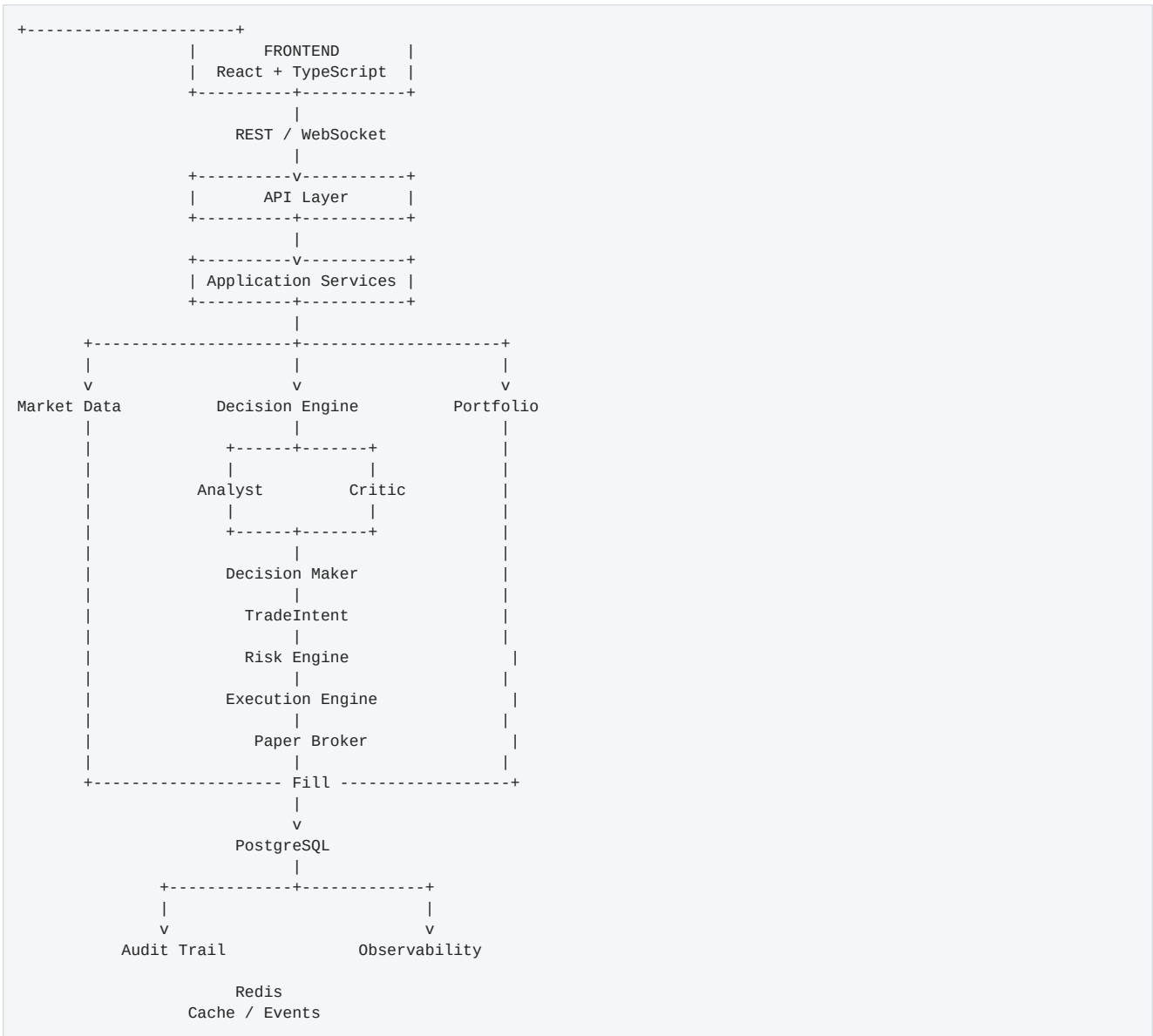
O sistema foi projetado para ambiente exclusivamente experimental/sandbox/paper. A arquitetura impede que a IA possua autoridade direta sobre execução e mantém o Risk Engine como barreira determinística.

O Blueprint está congelado. Qualquer mudança arquitetural exige nova versão e ADR.

## Princípios Arquiteturais

- LLM nunca possui autoridade direta de execução.
- Risk Engine é determinístico.
- V1 não possui Live Broker.
- PostgreSQL é a fonte de verdade.
- Redis é cache/coordenação e não fonte de verdade financeira.
- Sistema opera em fail-closed.
- Stops e proteção de posições não dependem da disponibilidade do LLM.
- UTC é o timezone interno.
- Cálculos financeiros críticos utilizam Decimal.
- Nenhuma decisão pode utilizar informação futura.
- Toda operação crítica deve ser idempotente.
- Toda operação importante deve possuir correlation\_id.
- Hard limits não podem ser alterados pela IA ou pela UI.
- Mudanças arquiteturais exigem ADR e nova versão do Blueprint.
- Requisito arquitetural não especificado deve gerar ARCHITECTURAL\_BLOCKER.

# ARQUITETURA



## Risk Model

Reference capital:  
R\$ 100,00

Maximum risk per trade:  
1%

Maximum simultaneous positions:  
1

Mandatory stop:  
YES

Circuit breaker:  
10% drawdown

V1 execution:  
PAPER / DEMO ONLY

Hard safety limits cannot be overridden by:

- LLM
- frontend
- user configuration
- provider response

## AI Architecture

MarketState

|

v

Analyst

|

v

Critic

|

v

Decision Maker

|

v

TradeIntent

|

v

Risk Engine

The LLM may:

- analyze market information
- produce structured reasoning
- suggest LONG / SHORT / HOLD / CLOSE
- suggest entry/SL/TP
- provide confidence
- provide thesis/invalidation

The LLM may NOT:

- call the broker
- submit orders
- change hard risk limits
- change kill switch
- access secrets
- bypass validation
- modify accounting

# DOMAIN CONTRACTS

## MarketState

```
market_state_id
asset
timestamp
timeframe
ohlcv
indicators
market_context
data_quality
source
hash
```

## TradeIntent

```
trade_intent_id
asset
action
quantity
entry_price
stop_loss
take_profit
confidence
thesis
invalidation
created_at
market_state_id
ai_run_id
```

## RiskDecision

```
risk_decision_id
trade_intent_id
status
approved_quantity
approved_price
risk_amount
exposure
reasons
created_at
```

## OrderRequest

```
execution_id
trade_intent_id
risk_decision_id
client_order_id
asset
side
type
quantity
price
stop_loss
take_profit
created_at
idempotency_key
```

## Order

```
order_id
client_order_id
status
filled_quantity
remaining_quantity
average_price
fees
created_at
updated_at
```

## Fill

```
fill_id
order_id
quantity
price
```

```
fee
timestamp
```

## Position

```
position_id
asset
side
quantity
average_entry
current_price
stop_loss
take_profit
realized_pnl
unrealized_pnl
opened_at
updated_at
```

## PortfolioSnapshot

```
snapshot_id
timestamp
cash
equity
exposure
realized_pnl
unrealized_pnl
drawdown
```

## AIRun

```
ai_run_id
agent
provider
model
prompt_version
input_hash
output_hash
started_at
completed_at
status
latency_ms
token_usage
```

## AuditEvent

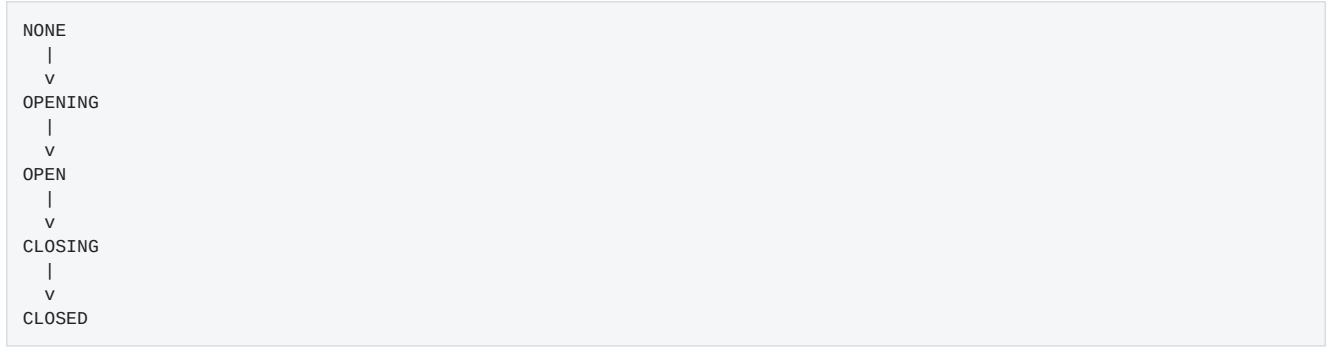
```
audit_event_id
correlation_id
event_type
entity_type
entity_id
timestamp
actor
payload_hash
```

# STATE MACHINES

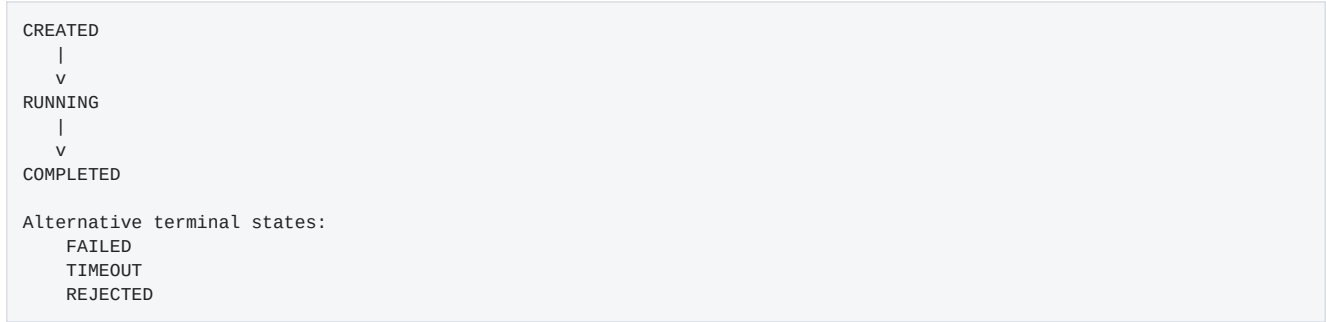
## Order



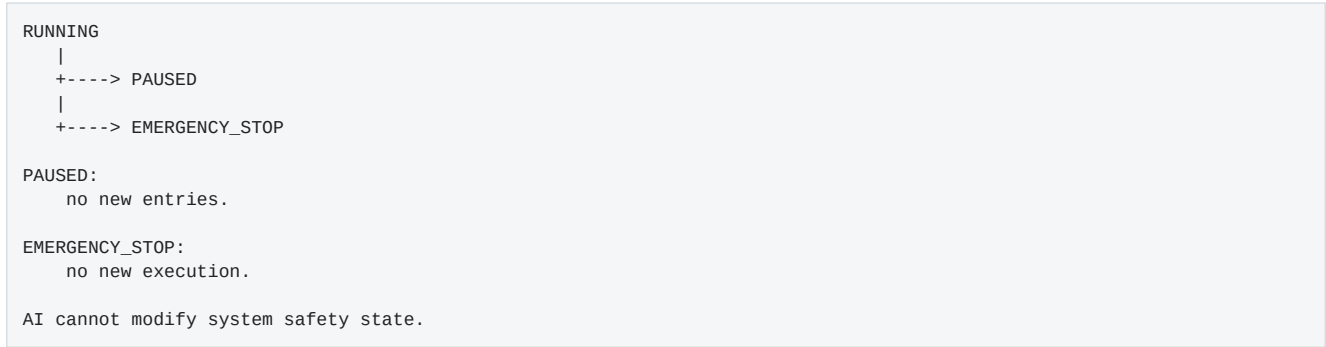
## Position



## AI Run



## System



# EVENT MODEL

Every important operation must carry:

```
correlation_id
event_time
ingestion_time
processing_time
```

When applicable:

```
execution_time
```

Typical event flow:

```
MarketDataReceived
->
CandleClosed
->
MarketStateCreated
->
AIRunStarted
->
AIRunCompleted
->
TradeIntentCreated
->
RiskDecisionCreated
->
OrderCreated
->
OrderSubmitted
->
OrderAcknowledged
->
FillReceived
->
PositionUpdated
->
PortfolioSnapshotCreated
->
AuditEventRecorded
```

No component may consume information whose event\_time is later than the MarketState timestamp used for a decision.



# CONFIGURATION HIERARCHY

Authority hierarchy:

1. HARD SAFETY LIMITS
2. APPLICATION CONFIGURATION
3. USER CONFIGURATION
4. AI SUGGESTION

Lower levels cannot override higher levels.

Examples of user-configurable settings:

- LLM provider
- model
- prompt version
- risk parameters within hard limits
- stop parameters within allowed range
- market universe
- fees/slippage for simulation

Examples of hard limits:

- maximum risk
- maximum position count
- emergency stop
- live broker disabled

# FAILURE & RECOVERY

Startup:

Application Start

->

Load Configuration

->

Database Connectivity

->

Load Portfolio

->

Reconcile Orders

->

Validate Positions

->

Validate Risk

->

Recover Workers

->

RUN or PAUSE

If state is ambiguous:

PAUSE

Never guess.

LLM unavailable:

- no new entries
- existing positions remain protected
- deterministic risk/protection continues

Market data inconsistent:

- no new decision

Risk Engine unavailable:

- no execution

Order state unknown:

- reconcile before continuing

# OBSERVABILITY

Technical metrics:

- API latency
- LLM latency

LLM token usage  
worker queue depth  
market data lag  
WebSocket connections  
DB latency  
Redis latency  
error rate

AI metrics:

invalid output rate  
timeout rate  
risk rejection rate  
LONG/HOLD/CLOSE distribution  
average confidence  
decision consistency

Trading metrics:

P&L  
drawdown  
win rate  
profit factor  
Sharpe  
exposure  
fees  
slippage

## DATASET & EXPERIMENT REGISTRY

Dataset:

dataset\_id  
source  
version  
symbols  
timeframe  
start  
end  
checksum  
created\_at

Experiment:

experiment\_id  
model  
prompt\_version  
dataset\_id  
configuration  
seed  
results  
created\_at

## SECURITY MODEL

Secrets:

- never committed to Git
- never returned to frontend
- never injected into LLM prompts
- never stored in audit payloads

V1:

- no live exchange credentials
- no LiveBroker implementation
- no production execution endpoint

API:

- authentication boundary
- authorization boundary
- input validation
- rate limiting where applicable

Frontend:

- cannot access provider secrets
- cannot bypass Risk Engine
- cannot directly create broker orders

## ADR GOVERNANCE

Directory:

```
docs/  
  adr/  
    ADR-001-ai-no-execution.md  
    ADR-002-paper-only.md  
    ADR-003-postgres-source-of-truth.md  
    ADR-004-deterministic-risk.md  
    ADR-005-utc.md  
    ADR-006-fail-closed.md
```

Process:

Proposal

->

ADR

->

Blueprint version

->

Implementation

No architectural decision may be silently introduced into code.

## DEFINITION OF DONE

A phase is complete only when:

```
[ ] implementation complete  
[ ] unit tests passing  
[ ] integration tests passing  
[ ] phase-specific tests passing  
[ ] architecture verified  
[ ] security verified  
[ ] audit prompt executed  
[ ] deviations corrected  
[ ] evidence collected  
[ ] Git commit recorded  
[ ] PHASE_XX_APPROVED.md created
```

Final V1:

V1\_STATUS = PAPER\_READY

NOT:

V1\_STATUS = LIVE\_READY

# IMPLEMENTATION PLAN V1.1 — 16 FASES

As 16 fases foram definidas por fronteiras arquiteturais e dependências reais. O número não é arbitrário e não deve ser aumentado apenas para criar granularidade artificial.

## FASE 01 — Foundation & Architecture Contracts

### Objetivo

Criar o esqueleto do projeto e materializar no código os contratos arquiteturais que serão utilizados por todas as demais fases.

### Escopo

Docker/Compose, estrutura do projeto, configuração base, Domain Contracts, enums, State Machines, Event Contracts, Time Contract, UTC, correlation\_id, idempotency conventions, estrutura de ADR e Architecture Freeze.

### Fora do Escopo

Banco de dados, trading, broker, IA operacional, frontend funcional e execução.

### Dependências

Nenhuma. É a fase inicial.

### Visão Arquitetural

Esta fase estabelece a linguagem formal do sistema. Os contratos devem ser independentes de infraestrutura sempre que possível.

O objetivo é impedir que fases posteriores inventem entidades, estados ou formatos incompatíveis.

### Critérios de Aceite

- Projeto sobe através do mecanismo de containerização definido.
- Domain Contracts existem e possuem validação.
- Enums oficiais estão centralizados.
- State Machines estão formalizadas e testadas.
- UTC é a convenção interna.
- Correlation ID e idempotency conventions estão definidas.
- ADR structure existe.
- ARCHITECTURAL\_BLOCKER está definido como mecanismo de governança.

### Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 01 – FOUNDATION & ARCHITECTURE CONTRACTS

=====

OBJETIVO:

Criar o esqueleto do projeto e materializar no código os contratos arquiteturais que serão utilizados por todas as demais fases.

ESCOPO:

Docker/Compose, estrutura do projeto, configuração base, Domain Contracts, enums, State Machines, Event Contracts, Time Contract, UTC, correlation\_id, idempotency conventions, estrutura de ADR e Architecture Freeze.

#### FORA DO ESCOPO:

Banco de dados, trading, broker, IA operacional, frontend funcional e execução.

#### DEPENDÊNCIAS:

Nenhuma. É a fase inicial.

#### VISÃO ARQUITETURAL:

Esta fase estabelece a linguagem formal do sistema. Os contratos devem ser independentes de infraestrutura sempre que possível.

O objetivo é impedir que fases posteriores inventem entidades, estados ou formatos incompatíveis.

#### CRITÉRIOS DE ACEITE:

- Projeto sobe através do mecanismo de containerização definido.
- Domain Contracts existem e possuem validação.
- Enums oficiais estão centralizados.
- State Machines estão formalizadas e testadas.
- UTC é a convenção interna.
- Correlation ID e idempotency conventions estão definidas.
- ADR structure existe.
- ARCHITECTURAL\_BLOCKER está definido como mecanismo de governança.

#### TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão. A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.

8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

**IMPORTANTE:**

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

**STATUS:**

PASS / FAIL / ARCHITECTURAL\_BLOCKER

**SUMMARY:**

resumo objetivo

**IMPLEMENTED:**

o que foi encontrado implementado

**MISSING:**

o que deveria existir e não existe

**OUT\_OF\_SCOPE:**

funcionalidades indevidas

**ARCHITECTURAL\_VIOLATIONS:**

violações encontradas

**TEST\_RESULTS:**

testes executados e resultado

**SECURITY\_RESULTS:**

resultado

**EVIDENCE:**

arquivos, testes e evidências

**RECOMMENDATION:**

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====  
AUDITORIA DA FASE 01 – FOUNDATION & ARCHITECTURE CONTRACTS  
=====

**OBJETIVO ESPERADO:**

Criar o esqueleto do projeto e materializar no código os contratos arquiteturais que serão utilizados por todas as demais fases.

**ESCOPO ESPERADO:**

Docker/Compose, estrutura do projeto, configuração base, Domain Contracts, enums, State Machines, Event Contracts, Time Contract, UTC, correlation\_id, idempotency conventions, estrutura de ADR e Architecture Freeze.

**FORA DO ESCOPO:**

Banco de dados, trading, broker, IA operacional, frontend

funcional e execução.

DEPENDÊNCIAS:

Nenhuma. É a fase inicial.

VISÃO ARQUITETURAL:

Esta fase estabelece a linguagem formal do sistema.  
Os contratos devem ser independentes de infraestrutura  
sempre que possível.

O objetivo é impedir que fases posteriores inventem  
entidades, estados ou formatos incompatíveis.

CRITÉRIOS DE ACEITE:

- Projeto sobre através do mecanismo de containerização definido.
- Domain Contracts existem e possuem validação.
- Enums oficiais estão centralizados.
- State Machines estão formalizadas e testadas.
- UTC é a convenção interna.
- Correlation ID e idempotency conventions estão definidas.
- ADR structure existe.
- ARCHITECTURAL\_BLOCKER está definido como mecanismo de governança.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes,  
critérios de aceite e auditoria estiverem completos,  
sem ARCHITECTURAL\_BLOCKER pendente.



## FASE 02 — Infrastructure & Persistence

### Objetivo

Implementar persistência e infraestrutura base sem criar uma segunda fonte de verdade.

### Escopo

PostgreSQL, Redis, migrations, repositories, persistência dos contratos, configuração de infraestrutura, backup e restore.

### Fora do Escopo

Estratégia, IA, execução real, dashboard completo.

### Dependências

Fase 01.

### Visão Arquitetural

PostgreSQL é a fonte de verdade para estado financeiro, ordens, fills, posições, decisões e auditoria.

Redis é utilizado apenas para cache, coordenação e dados efêmeros.

### Critérios de Aceite

- PostgreSQL inicializa corretamente.
- Migrations são reproduzíveis.
- Repositories possuem testes.
- Redis funciona sem ser usado como source of truth.
- Backup é produzido.
- Restore foi efetivamente testado.

### Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você **NÃO** é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.

11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 02 – INFRASTRUCTURE & PERSISTENCE

=====

OBJETIVO:

Implementar persistência e infraestrutura base sem criar uma segunda fonte de verdade.

ESCOPO:

PostgreSQL, Redis, migrations, repositories, persistência dos contratos, configuração de infraestrutura, backup e restore.

FORA DO ESCOPO:

Estratégia, IA, execução real, dashboard completo.

DEPENDÊNCIAS:

Fase 01.

VISÃO ARQUITETURAL:

PostgreSQL é a fonte de verdade para estado financeiro, ordens, fills, posições, decisões e auditoria.

Redis é utilizado apenas para cache, coordenação e dados efêmeros.

CRITÉRIOS DE ACEITE:

- PostgreSQL inicializa corretamente.
- Migrations são reproduzíveis.
- Repositories possuem testes.
- Redis funciona sem ser usado como source of truth.

- Backup é produzido.
- Restore foi efetivamente testado.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:  
PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:  
resumo objetivo

IMPLEMENTED:  
o que foi encontrado implementado

MISSING:  
o que deveria existir e não existe

OUT\_OF\_SCOPE:  
funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:  
violações encontradas

TEST\_RESULTS:  
testes executados e resultado

SECURITY\_RESULTS:  
resultado

EVIDENCE:  
arquivos, testes e evidências

RECOMMENDATION:  
APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 02 – INFRASTRUCTURE & PERSISTENCE

=====

OBJETIVO ESPERADO:

Implementar persistência e infraestrutura base sem criar uma segunda fonte de verdade.

ESCOPO ESPERADO:

PostgreSQL, Redis, migrations, repositories, persistência dos contratos, configuração de infraestrutura, backup e restore.

FORA DO ESCOPO:

Estratégia, IA, execução real, dashboard completo.

DEPENDÊNCIAS:

Fase 01.

VISÃO ARQUITETURAL:

PostgreSQL é a fonte de verdade para estado financeiro, ordens, fills, posições, decisões e auditoria.

Redis é utilizado apenas para cache, coordenação e dados efêmeros.

CRITÉRIOS DE ACEITE:

- PostgreSQL inicializa corretamente.
- Migrations são reproduzíveis.
- Repositories possuem testes.
- Redis funciona sem ser usado como source of truth.
- Backup é produzido.
- Restore foi efetivamente testado.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturas.
7. Procure riscos de segurança.

8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 03 — Backend Core & Application Runtime

## Objetivo

Criar o runtime principal da aplicação.

## Escopo

API, application services, dependency injection, structured logging, error handling, health checks, configuration, workers, scheduler e estados RUNNING/PAUSED/EMERGENCY\_STOP.

## Fora do Escopo

Decision Engine, LLM operacional, Paper Broker.

## Dependências

Fases 01 e 02.

## Visão Arquitetural

O backend deve possuir separação clara entre domínio, aplicação e infraestrutura.

O runtime precisa suportar restart e recuperação sem reconstruir estado por inferência.

## Crítérios de Aceite

- Backend inicia e responde health checks.
- Workers possuem ciclo de vida controlado.
- Structured logging está ativo.
- RUNNING/PAUSED/EMERGENCY\_STOP funcionam.
- Restart não corrompe estado persistido.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.

13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 03 – BACKEND CORE & APPLICATION RUNTIME

=====

OBJETIVO:

Criar o runtime principal da aplicação.

ESCOPO:

API, application services, dependency injection, structured logging, error handling, health checks, configuration, workers, scheduler e estados RUNNING/PAUSED/EMERGENCY\_STOP.

FORA DO ESCOPO:

Decision Engine, LLM operacional, Paper Broker.

DEPENDÊNCIAS:

Fases 01 e 02.

VISÃO ARQUITETURAL:

O backend deve possuir separação clara entre domínio, aplicação e infraestrutura.

O runtime precisa suportar restart e recuperação sem reconstruir estado por inferência.

CRITÉRIOS DE ACEITE:

- Backend inicia e responde health checks.
- Workers possuem ciclo de vida controlado.
- Structured logging está ativo.
- RUNNING/PAUSED/EMERGENCY\_STOP funcionam.
- Restart não corrompe estado persistido.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER



SUMMARY:  
resumo objetivo

IMPLEMENTED:  
o que foi encontrado implementado

MISSING:  
o que deveria existir e não existe

OUT\_OF\_SCOPE:  
funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:  
violações encontradas

TEST\_RESULTS:  
testes executados e resultado

SECURITY\_RESULTS:  
resultado

EVIDENCE:  
arquivos, testes e evidências

RECOMMENDATION:  
APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 03 – BACKEND CORE & APPLICATION RUNTIME

=====

OBJETIVO ESPERADO:

Criar o runtime principal da aplicação.

ESCOPO ESPERADO:

API, application services, dependency injection, structured logging, error handling, health checks, configuration, workers, scheduler e estados RUNNING/PAUSED/EMERGENCY\_STOP.

FORA DO ESCOPO:

Decision Engine, LLM operacional, Paper Broker.

DEPENDÊNCIAS:

Fases 01 e 02.

VISÃO ARQUITETURAL:

O backend deve possuir separação clara entre domínio, aplicação e infraestrutura.

O runtime precisa suportar restart e recuperação sem reconstruir estado por inferência.

CRITÉRIOS DE ACEITE:

- Backend inicia e responde health checks.
- Workers possuem ciclo de vida controlado.
- Structured logging está ativo.
- RUNNING/PAUSED/EMERGENCY\_STOP funcionam.
- Restart não corrompe estado persistido.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.

12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 04 — Market Data Engine

## Objetivo

Implementar ingestão, normalização e persistência dos dados de mercado.

## Escopo

Provider Adapter, historical data, candles, normalization, closed-candle detection, gaps, deduplicação, cache e persistência.

## Fora do Escopo

Decisão de trading e execução.

## Dependências

Fases 01-03.

## Visão Arquitetural

O Market Data Engine deve produzir dados temporais auditáveis e impedir look-ahead.

Apenas candles CLOSED podem alimentar decisões.

## CrITÉRIOS de Aceite

- Candles são normalizados.
- Eventos duplicados são tratados.
- Gaps são detectados.
- Candle aberto não alimenta decisão.
- Timestamp é tratado em UTC.
- Não existe look-ahead.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.

11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 04 – MARKET DATA ENGINE

=====

OBJETIVO:

Implementar ingestão, normalização e persistência dos dados de mercado.

ESCOPO:

Provider Adapter, historical data, candles, normalization, closed-candle detection, gaps, deduplicação, cache e persistência.

FORA DO ESCOPO:

Decisão de trading e execução.

DEPENDÊNCIAS:

Fases 01-03.

VISÃO ARQUITETURAL:

O Market Data Engine deve produzir dados temporais auditáveis e impedir look-ahead.

Apenas candles CLOSED podem alimentar decisões.

CRITÉRIOS DE ACEITE:

- Candles são normalizados.
- Eventos duplicados são tratados.
- Gaps são detectados.
- Candle aberto não alimenta decisão.

- Timestamp é tratado em UTC.
- Não existe look-ahead.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:  
PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:  
resumo objetivo

IMPLEMENTED:  
o que foi encontrado implementado

MISSING:  
o que deveria existir e não existe

OUT\_OF\_SCOPE:  
funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:  
violações encontradas

TEST\_RESULTS:  
testes executados e resultado

SECURITY\_RESULTS:  
resultado

EVIDENCE:  
arquivos, testes e evidências

RECOMMENDATION:  
APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 04 – MARKET DATA ENGINE

=====

OBJETIVO ESPERADO:

Implementar ingestão, normalização e persistência dos dados de mercado.

ESCOPO ESPERADO:

Provider Adapter, historical data, candles, normalization, closed-candle detection, gaps, deduplicação, cache e persistência.

FORA DO ESCOPO:

Decisão de trading e execução.

DEPENDÊNCIAS:

Fases 01-03.

VISÃO ARQUITETURAL:

O Market Data Engine deve produzir dados temporais auditáveis e impedir look-ahead.

Apenas candles CLOSED podem alimentar decisões.

CRITÉRIOS DE ACEITE:

- Candles são normalizados.
- Eventos duplicados são tratados.
- Gaps são detectados.
- Candle aberto não alimenta decisão.
- Timestamp é tratado em UTC.
- Não existe look-ahead.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturas.
7. Procure riscos de segurança.

8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 05 — Universe & Market State

## Objetivo

Criar a seleção de ativos e o MarketState formal.

## Escopo

Universe Scanner, critérios de elegibilidade, liquidez, volume, spread, disponibilidade, qualidade dos dados e construção do MarketState.

## Fora do Escopo

Ordem, broker, IA de decisão.

## Dependências

Fase 04.

## Visão Arquitetural

Universe define quais ativos podem ser analisados.

MarketState representa somente informações disponíveis naquele instante.

## Crítérios de Aceite

- Universe é determinístico.
- MarketState possui hash.
- MarketState referencia fonte e timestamp.
- Dados futuros são rejeitados.
- Ativos sem dados suficientes não entram no pipeline.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.



16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 05 – UNIVERSE & MARKET STATE

=====

OBJETIVO:

Criar a seleção de ativos e o MarketState formal.

ESCOPO:

Universe Scanner, critérios de elegibilidade, liquidez, volume, spread, disponibilidade, qualidade dos dados e construção do MarketState.

FORA DO ESCOPO:

Ordem, broker, IA de decisão.

DEPENDÊNCIAS:

Fase 04.

VISÃO ARQUITETURAL:

Universe define quais ativos podem ser analisados.  
MarketState representa somente informações disponíveis naquele instante.

CRITÉRIOS DE ACEITE:

- Universe é determinístico.
- MarketState possui hash.
- MarketState referencia fonte e timestamp.
- Dados futuros são rejeitados.
- Ativos sem dados suficientes não entram no pipeline.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.

A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:  
o que deveria existir e não existe

OUT\_OF\_SCOPE:  
funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:  
violações encontradas

TEST\_RESULTS:  
testes executados e resultado

SECURITY\_RESULTS:  
resultado

EVIDENCE:  
arquivos, testes e evidências

RECOMMENDATION:  
APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 05 – UNIVERSE & MARKET STATE

=====

#### OBJETIVO ESPERADO:

Criar a seleção de ativos e o MarketState formal.

#### ESCOPO ESPERADO:

Universe Scanner, critérios de elegibilidade, liquidez, volume, spread, disponibilidade, qualidade dos dados e construção do MarketState.

#### FORA DO ESCOPO:

Ordem, broker, IA de decisão.

DEPENDÊNCIAS:  
Fase 04.

#### VISÃO ARQUITETURAL:

Universe define quais ativos podem ser analisados.  
MarketState representa somente informações disponíveis naquele instante.

#### CRITÉRIOS DE ACEITE:

- Universe é determinístico.
- MarketState possui hash.
- MarketState referencia fonte e timestamp.
- Dados futuros são rejeitados.
- Ativos sem dados suficientes não entram no pipeline.

#### PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 06 — Paper Broker & Execution

## Objetivo

Implementar a execução simulada e a máquina de estados de ordens.

## Escopo

TradeIntent validation, Execution Engine, Paper Broker, MARKET/LIMIT, fills, partial fills, fees, spread, slippage, cancelamento, rejeição, retries e idempotência.

## Fora do Escopo

Live Broker.

## Dependências

Fases 01-05.

## Visão Arquitetural

TradeIntent não é Order.

O fluxo obrigatório é:

TradeIntent  
->  
Execution Validation  
->  
Execution Engine  
->  
Paper Broker  
->  
Order/Fill

## Crítérios de Aceite

- Order State Machine é respeitada.
- Duplicate request não gera segunda ordem.
- Retry não duplica execução.
- Partial fill é tratado.
- Fees e slippage são registrados.
- Nenhum LiveBroker existe.

## Prompt de Implementação

```
Você está implementando uma fase do projeto Autonomous Swing Trading Bot.  
  
A autoridade arquitetural é exclusivamente:  
  
    BLUEPRINT V1.1  
  
Status:  
  
    ARCHITECTURE FROZEN  
  
Você NÃO é o arquiteto.  
  
Você deve implementar exatamente o escopo da fase solicitada.
```

#### REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

#### ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

#### DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

#### AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 06 – PAPER BROKER & EXECUTION

=====

#### OBJETIVO:

Implementar a execução simulada e a máquina de estados de ordens.

#### ESCOPO:

TradeIntent validation, Execution Engine, Paper Broker, MARKET/LIMIT, fills, partial fills, fees, spread, slippage, cancelamento, rejeição, retries e idempotência.

#### FORA DO ESCOPO:

Live Broker.

#### DEPENDÊNCIAS:

Fases 01-05.

#### VISÃO ARQUITETURAL:

TradeIntent não é Order.

O fluxo obrigatório é:

```
TradeIntent
->
Execution Validation
->
Execution Engine
->
Paper Broker
->
Order/Fill
```

#### CRITÉRIOS DE ACEITE:

- Order State Machine é respeitada.
- Duplicate request não gera segunda ordem.
- Retry não duplica execução.
- Partial fill é tratado.
- Fees e slippage são registrados.
- Nenhum LiveBroker existe.

#### TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.

14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 06 – PAPER BROKER & EXECUTION

=====

OBJETIVO ESPERADO:

Implementar a execução simulada e a máquina de estados de ordens.

ESCOPO ESPERADO:

TradeIntent validation, Execution Engine, Paper Broker, MARKET/LIMIT, fills, partial fills, fees, spread, slippage, cancelamento, rejeição, retries e idempotência.

FORA DO ESCOPO:

Live Broker.

DEPENDÊNCIAS:

Fases 01-05.

VISÃO ARQUITETURAL:

TradeIntent não é Order.



O fluxo obrigatório é:

```
TradeIntent
->
Execution Validation
->
Execution Engine
->
Paper Broker
->
Order/Fill
```

#### CRITÉRIOS DE ACEITE:

- Order State Machine é respeitada.
- Duplicate request não gera segunda ordem.
- Retry não duplica execução.
- Partial fill é tratado.
- Fees e slippage são registrados.
- Nenhum LiveBroker existe.

#### PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 07 — Portfolio & Accounting

## Objetivo

Implementar contabilidade determinística do portfólio.

## Escopo

Cash, equity, positions, average entry, realized P&L, unrealized P&L, fees, exposure e drawdown.

## Fora do Escopo

IA, estratégia e dashboard completo.

## Dependências

Fase 06.

## Visão Arquitetural

Fill é a fonte do evento de alteração de posição.  
Accounting utiliza Decimal para cálculos financeiros críticos.

## Critérios de Aceite

- Cálculos utilizam Decimal.
- Compra funciona.
- Venda funciona.
- Partial fill funciona.
- Fees são contabilizados.
- P&L realizado funciona.
- P&L não realizado funciona.
- Posição zerada é tratada corretamente.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.

13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 07 – PORTFOLIO & ACCOUNTING

=====

OBJETIVO:

Implementar contabilidade determinística do portfólio.

ESCOPO:

Cash, equity, positions, average entry, realized P&L, unrealized P&L, fees, exposure e drawdown.

FORA DO ESCOPO:

IA, estratégia e dashboard completo.

DEPENDÊNCIAS:

Fase 06.

VISÃO ARQUITETURAL:

Fill é a fonte do evento de alteração de posição.  
Accounting utiliza Decimal para cálculos financeiros críticos.

CRITÉRIOS DE ACEITE:

- Cálculos utilizam Decimal.
- Compra funciona.
- Venda funciona.
- Partial fill funciona.
- Fees são contabilizados.
- P&L realizado funciona.
- P&L não realizado funciona.
- Posição zerada é tratada corretamente.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 07 – PORTFOLIO & ACCOUNTING

=====

OBJETIVO ESPERADO:

Implementar contabilidade determinística do portfólio.

ESCOPO ESPERADO:

Cash, equity, positions, average entry, realized P&L,  
unrealized P&L, fees, exposure e drawdown.

FORA DO ESCOPO:

IA, estratégia e dashboard completo.

DEPENDÊNCIAS:

Fase 06.

VISÃO ARQUITETURAL:

Fill é a fonte do evento de alteração de posição.  
Accounting utiliza Decimal para cálculos financeiros críticos.

CRITÉRIOS DE ACEITE:

- Cálculos utilizam Decimal.
- Compra funciona.
- Venda funciona.
- Partial fill funciona.
- Fees são contabilizados.
- P&L realizado funciona.
- P&L não realizado funciona.
- Posição zerada é tratada corretamente.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.

14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 08 — Risk Engine & Safety

## Objetivo

Criar a barreira determinística de risco.

## Escopo

Position sizing, max risk, max positions, mandatory stop, exposure, circuit breaker, capital validation, tick size, step size, min notional, kill switch, hard limits e fail-closed.

## Fora do Escopo

LLM, estratégia e execução direta.

## Dependências

Fases 01, 06 e 07.

## Visão Arquitetural

TradeIntent entra no Risk Engine e sai como RiskDecision.

O Risk Engine pode rejeitar qualquer decisão.

IA e frontend não podem ultrapassar seus limites.

## Crítérios de Aceite

- Risco máximo por operação: 1%.
- Máximo de posições: 1.
- Stop obrigatório.
- Circuit breaker de 10% configurado.
- Hard limits não podem ser sobrescritos.
- Risk Engine indisponível implica NO EXECUTION.
- Testes de bypass falham de forma segura.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.

9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 08 – RISK ENGINE & SAFETY

=====

OBJETIVO:

Criar a barreira determinística de risco.

ESCOPO:

Position sizing, max risk, max positions, mandatory stop, exposure, circuit breaker, capital validation, tick size, step size, min notional, kill switch, hard limits e fail-closed.

FORA DO ESCOPO:

LLM, estratégia e execução direta.

DEPENDÊNCIAS:

Fases 01, 06 e 07.

VISÃO ARQUITETURAL:

TradeIntent entra no Risk Engine e sai como RiskDecision.

O Risk Engine pode rejeitar qualquer decisão.

IA e frontend não podem ultrapassar seus limites.

CRITÉRIOS DE ACEITE:

- Risco máximo por operação: 1%.
- Máximo de posições: 1.



- Stop obrigatório.
- Circuit breaker de 10% configurado.
- Hard limits não podem ser sobrescritos.
- Risk Engine indisponível implica NO EXECUTION.
- Testes de bypass falham de forma segura.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.  
Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 08 – RISK ENGINE & SAFETY

=====

OBJETIVO ESPERADO:

Criar a barreira determinística de risco.

ESCOPO ESPERADO:

Position sizing, max risk, max positions, mandatory stop,  
exposure, circuit breaker, capital validation, tick size,  
step size, min notional, kill switch, hard limits e fail-closed.

FORA DO ESCOPO:

LLM, estratégia e execução direta.

DEPENDÊNCIAS:

Fases 01, 06 e 07.

VISÃO ARQUITETURAL:

TradeIntent entra no Risk Engine e sai como RiskDecision.

O Risk Engine pode rejeitar qualquer decisão.

IA e frontend não podem ultrapassar seus limites.

CRITÉRIOS DE ACEITE:

- Risco máximo por operação: 1%.
- Máximo de posições: 1.
- Stop obrigatório.
- Circuit breaker de 10% configurado.
- Hard limits não podem ser sobrescritos.
- Risk Engine indisponível implica NO EXECUTION.
- Testes de bypass falham de forma segura.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.

4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique `correlation_id` quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

`STATUS = FAIL`

Se houver requisito arquitetural não especificado:

`STATUS = ARCHITECTURAL_BLOCKER`

Somente produza:

`STATUS = PASS`

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 09 — AI Gateway

## Objetivo

Criar a camada de integração com providers de LLM sem permitir autoridade operacional direta.

## Escopo

Provider Adapter, modelos, credentials, timeout, retry, structured output, schema validation, prompt versioning, token accounting, mock provider e AI Run.

## Fora do Escopo

Decision Pipeline completo e execução.

## Dependências

Fases 01-08.

## Visão Arquitetural

O AI Gateway é a única fronteira de integração com o LLM.

Nenhum provider recebe acesso ao broker, banco financeiro ou kill switch.

## CrITÉrios de Aceite

- Provider pode ser substituído por adapter.
- Output inválido é rejeitado.
- Timeout é tratado.
- Retry possui limites.
- Secrets não chegam ao frontend nem ao prompt.
- AI Run é auditável.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.

11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 09 – AI GATEWAY

=====

OBJETIVO:

Criar a camada de integração com providers de LLM sem permitir autoridade operacional direta.

ESCOPO:

Provider Adapter, modelos, credentials, timeout, retry, structured output, schema validation, prompt versioning, token accounting, mock provider e AI Run.

FORA DO ESCOPO:

Decision Pipeline completo e execução.

DEPENDÊNCIAS:

Fases 01-08.

VISÃO ARQUITETURAL:

O AI Gateway é a única fronteira de integração com o LLM.

Nenhum provider recebe acesso ao broker, banco financeiro ou kill switch.

CRITÉRIOS DE ACEITE:

- Provider pode ser substituído por adapter.
- Output inválido é rejeitado.
- Timeout é tratado.
- Retry possui limites.

- Secrets não chegam ao frontend nem ao prompt.
- AI Run é auditável.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:  
PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:  
resumo objetivo

IMPLEMENTED:  
o que foi encontrado implementado

MISSING:  
o que deveria existir e não existe

OUT\_OF\_SCOPE:  
funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:  
violações encontradas

TEST\_RESULTS:  
testes executados e resultado

SECURITY\_RESULTS:  
resultado

EVIDENCE:  
arquivos, testes e evidências

RECOMMENDATION:  
APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 09 – AI GATEWAY

=====

OBJETIVO ESPERADO:

Criar a camada de integração com providers de LLM sem permitir autoridade operacional direta.

ESCOPO ESPERADO:

Provider Adapter, modelos, credentials, timeout, retry, structured output, schema validation, prompt versioning, token accounting, mock provider e AI Run.

FORA DO ESCOPO:

Decision Pipeline completo e execução.

DEPENDÊNCIAS:

Fases 01-08.

VISÃO ARQUITETURAL:

O AI Gateway é a única fronteira de integração com o LLM.

Nenhum provider recebe acesso ao broker, banco financeiro ou kill switch.

CRITÉRIOS DE ACEITE:

- Provider pode ser substituído por adapter.
- Output inválido é rejeitado.
- Timeout é tratado.
- Retry possui limites.
- Secrets não chegam ao frontend nem ao prompt.
- AI Run é auditável.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.

8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.



# FASE 10 — AI Agents & Decision Pipeline

## Objetivo

Implementar Analyst, Critic e Decision Maker.

## Escopo

MarketState -> Analyst -> Critic -> Decision Maker -> TradeIntent, incluindo confidence, thesis, invalidation, entry, stop e target quando aplicável.

## Fora do Escopo

Broker direto e bypass do Risk Engine.

## Dependências

Fases 05, 08 e 09.

## Visão Arquitetural

A IA produz intenção estruturada.

Ela nunca produz uma Order diretamente.

AI:

MarketState -> TradeIntent

Risk:

TradeIntent -> RiskDecision

## Critérios de Aceite

- Analyst funciona.
- Critic funciona.
- Decision Maker produz schema válido.
- TradeIntent possui ai\_run\_id.
- Prompt version é registrada.
- Input/output hashes são registrados.
- Não existe caminho AI -> Broker.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.

3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

```
=====
FASE 10 – AI AGENTS & DECISION PIPELINE
=====
```

OBJETIVO:

Implementar Analyst, Critic e Decision Maker.

ESCOPO:

MarketState -> Analyst -> Critic -> Decision Maker ->  
TradeIntent, incluindo confidence, thesis, invalidation,  
entry, stop e target quando aplicável.

FORA DO ESCOPO:

Broker direto e bypass do Risk Engine.

DEPENDÊNCIAS:

Fases 05, 08 e 09.

VISÃO ARQUITETURAL:

A IA produz intenção estruturada.

Ela nunca produz uma Order diretamente.

AI:  
MarketState -> TradeIntent

Risk:  
TradeIntent -> RiskDecision

CRITÉRIOS DE ACEITE:

- Analyst funciona.
- Critic funciona.
- Decision Maker produz schema válido.
- TradeIntent possui ai\_run\_id.
- Prompt version é registrada.
- Input/output hashes são registrados.
- Não existe caminho AI -> Broker.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.  
Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 10 – AI AGENTS & DECISION PIPELINE

=====

OBJETIVO ESPERADO:

Implementar Analyst, Critic e Decision Maker.

ESCOPO ESPERADO:

MarketState -> Analyst -> Critic -> Decision Maker ->  
TradeIntent, incluindo confidence, thesis, invalidation,  
entry, stop e target quando aplicável.

FORA DO ESCOPO:

Broker direto e bypass do Risk Engine.

DEPENDÊNCIAS:

Fases 05, 08 e 09.

VISÃO ARQUITETURAL:

A IA produz intenção estruturada.

Ela nunca produz uma Order diretamente.

AI:

MarketState -> TradeIntent

Risk:

TradeIntent -> RiskDecision

CRITÉRIOS DE ACEITE:

- Analyst funciona.
- Critic funciona.
- Decision Maker produz schema válido.
- TradeIntent possui ai\_run\_id.
- Prompt version é registrada.
- Input/output hashes são registrados.
- Não existe caminho AI -> Broker.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 11 — Decision + Risk + Execution Integration

## Objetivo

Integrar o pipeline completo de decisão e execução simulada.

## Escopo

Orchestration, correlation ID, idempotência, retries, failure handling e fluxo completo:  
MarketState -> AI -> TradeIntent -> Risk -> Execution -> Paper Broker -> Fill -> Portfolio.

## Fora do Escopo

Live trading.

## Dependências

Fases 06-10.

## Visão Arquitetural

Esta fase cria o coração operacional da V1.

Nenhuma camada pode saltar a Risk Engine.

## Critérios de Aceite

- E2E de operação completa funciona.
- Cada etapa possui correlation\_id.
- Falha em qualquer etapa produz estado seguro.
- Retry não duplica operação.
- AI failure bloqueia nova entrada.
- Proteção de posição não depende do LLM.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.

13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 11 – DECISION + RISK + EXECUTION INTEGRATION

=====

OBJETIVO:

Integrar o pipeline completo de decisão e execução simulada.

ESCOPO:

Orchestration, correlation ID, idempotência, retries,  
failure handling e fluxo completo:  
MarketState -> AI -> TradeIntent -> Risk -> Execution ->  
Paper Broker -> Fill -> Portfolio.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 06-10.

VISÃO ARQUITETURAL:

Esta fase cria o coração operacional da V1.

Nenhuma camada pode saltar a Risk Engine.

CRITÉRIOS DE ACEITE:

- E2E de operação completa funciona.
- Cada etapa possui correlation\_id.
- Falha em qualquer etapa produz estado seguro.
- Retry não duplica operação.
- AI failure bloqueia nova entrada.
- Proteção de posição não depende do LLM.

#### TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER



SUMMARY:  
resumo objetivo

IMPLEMENTED:  
o que foi encontrado implementado

MISSING:  
o que deveria existir e não existe

OUT\_OF\_SCOPE:  
funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:  
violações encontradas

TEST\_RESULTS:  
testes executados e resultado

SECURITY\_RESULTS:  
resultado

EVIDENCE:  
arquivos, testes e evidências

RECOMMENDATION:  
APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 11 – DECISION + RISK + EXECUTION INTEGRATION

=====

OBJETIVO ESPERADO:

Integrar o pipeline completo de decisão e execução simulada.

ESCOPO ESPERADO:

Orchestration, correlation ID, idempotência, retries, failure handling e fluxo completo:  
MarketState -> AI -> TradeIntent -> Risk -> Execution -> Paper Broker -> Fill -> Portfolio.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:  
Fases 06-10.

VISÃO ARQUITETURAL:

Esta fase cria o coração operacional da V1.

Nenhuma camada pode saltar a Risk Engine.

CRITÉRIOS DE ACEITE:

- E2E de operação completa funciona.
- Cada etapa possui correlation\_id.
- Falha em qualquer etapa produz estado seguro.
- Retry não duplica operação.
- AI failure bloqueia nova entrada.
- Proteção de posição não depende do LLM.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.

12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 12 — Replay & Backtest

## Objetivo

Criar o laboratório determinístico para replay e backtest.

## Escopo

Dataset Registry, Experiment Registry, replay engine, backtest, seed, configuração versionada, fees, spread, slippage, métricas e prevenção de look-ahead.

## Fora do Escopo

Produção/live trading.

## Dependências

Fases 04-11.

## Visão Arquitetural

Replay utiliza o mesmo pipeline de decisão sempre que possível, sem criar uma segunda implementação da estratégia.

Cada experimento identifica exatamente:

dataset  
model  
prompt  
configuration  
seed

## Critérios de Aceite

- Dataset possui checksum.
- Experiment possui identificação completa.
- Replay é reproduzível.
- Mesmo input + seed produz mesmo resultado.
- Look-ahead é impossível.
- Fees/slippage podem ser simulados.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.

6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 12 – REPLAY & BACKTEST

=====

OBJETIVO:

Criar o laboratório determinístico para replay e backtest.

ESCOPO:

Dataset Registry, Experiment Registry, replay engine, backtest, seed, configuração versionada, fees, spread, slippage, métricas e prevenção de look-ahead.

FORA DO ESCOPO:

Produção/live trading.

DEPENDÊNCIAS:

Fases 04-11.

VISÃO ARQUITETURAL:

Replay utiliza o mesmo pipeline de decisão sempre que possível, sem criar uma segunda implementação da estratégia.

Cada experimento identifica exatamente:

- dataset
- model
- prompt

configuration  
seed

CRITÉRIOS DE ACEITE:

- Dataset possui checksum.
- Experiment possui identificação completa.
- Replay é reproduzível.
- Mesmo input + seed produz mesmo resultado.
- Look-ahead é impossível.
- Fees/slippage podem ser simulados.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 12 – REPLAY & BACKTEST

=====

OBJETIVO ESPERADO:

Criar o laboratório determinístico para replay e backtest.

ESCOPO ESPERADO:

Dataset Registry, Experiment Registry, replay engine, backtest, seed, configuração versionada, fees, spread, slippage, métricas e prevenção de look-ahead.

FORA DO ESCOPO:

Produção/live trading.

DEPENDÊNCIAS:

Fases 04-11.

VISÃO ARQUITETURAL:

Replay utiliza o mesmo pipeline de decisão sempre que possível, sem criar uma segunda implementação da estratégia.

Cada experimento identifica exatamente:

dataset  
model  
prompt  
configuration  
seed

CRITÉRIOS DE ACEITE:

- Dataset possui checksum.
- Experiment possui identificação completa.
- Replay é reproduzível.

- Mesmo input + seed produz mesmo resultado.
- Look-ahead é impossível.
- Fees/slippage podem ser simulados.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 13 — Frontend & Operational Dashboard

## Objetivo

Criar a interface operacional completa.

## Escopo

Dashboard, posições, ordens, portfolio, performance, AI decisions, risk, market, configuration, providers, LLM, prompts, audit, backtest e system status.

## Fora do Escopo

Bypass de backend, Risk Engine ou broker.

## Dependências

Fases 02-12.

## Visão Arquitetural

Frontend é cliente.

Toda regra de segurança permanece no backend.

WebSocket fornece estado realtime, enquanto REST fornece operações e consultas adequadas.

## CrITÉrios de Aceite

- Posições abertas aparecem corretamente.
- P&L é consistente com backend.
- Ordens são exibidas.
- Risk status é visível.
- Kill switch é acessível segundo autorização.
- Secrets nunca aparecem.
- Frontend não consegue enviar Order diretamente ao broker.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.



8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplemente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 13 – FRONTEND & OPERATIONAL DASHBOARD

=====

OBJETIVO:

Criar a interface operacional completa.

ESCOPO:

Dashboard, posições, ordens, portfolio, performance,  
AI decisions, risk, market, configuration, providers,  
LLM, prompts, audit, backtest e system status.

FORA DO ESCOPO:

Bypass de backend, Risk Engine ou broker.

DEPENDÊNCIAS:

Fases 02-12.

VISÃO ARQUITETURAL:

Frontend é cliente.

Toda regra de segurança permanece no backend.

WebSocket fornece estado realtime, enquanto REST fornece  
operações e consultas adequadas.

CRITÉRIOS DE ACEITE:

- Posições abertas aparecem corretamente.
- P&L é consistente com backend.
- Ordens são exibidas.
- Risk status é visível.
- Kill switch é acessível segundo autorização.
- Secrets nunca aparecem.
- Frontend não consegue enviar Order diretamente ao broker.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 13 – FRONTEND & OPERATIONAL DASHBOARD

=====

OBJETIVO ESPERADO:

Criar a interface operacional completa.

ESCOPO ESPERADO:

Dashboard, posições, ordens, portfolio, performance,  
AI decisions, risk, market, configuration, providers,  
LLM, prompts, audit, backtest e system status.

FORA DO ESCOPO:

Bypass de backend, Risk Engine ou broker.

DEPENDÊNCIAS:

Fases 02-12.

VISÃO ARQUITETURAL:

Frontend é cliente.

Toda regra de segurança permanece no backend.

WebSocket fornece estado realtime, enquanto REST fornece  
operações e consultas adequadas.

CRITÉRIOS DE ACEITE:

- Posições abertas aparecem corretamente.
- P&L é consistente com backend.
- Ordens são exibidas.
- Risk status é visível.
- Kill switch é acessível segundo autorização.
- Secrets nunca aparecem.
- Frontend não consegue enviar Order diretamente ao broker.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 14 — Audit & Observability

## Objetivo

Consolidar rastreabilidade completa e métricas.

## Escopo

Audit Trail, structured logs, correlation chains, system metrics, AI metrics e trading metrics.

## Fora do Escopo

Alteração de regras de negócio.

## Dependências

Fases 01-13.

## Visão Arquitetural

Toda operação deve ser reconstruível através da cadeia:

MarketState

->

AI Run

->

TradeIntent

->

RiskDecision

->

Order

->

Fill

->

Position

->

Portfolio

## Critérios de Aceite

- Toda operação possui correlation\_id.
- Audit events são persistidos.
- Logs são estruturados.
- Métricas técnicas existem.
- Métricas de IA existem.
- Métricas de trading existem.
- Uma operação pode ser reconstruída integralmente.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

## ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

### REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

### ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

### DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

### AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

## FASE 14 – AUDIT & OBSERVABILITY

### OBJETIVO:

Consolidar rastreabilidade completa e métricas.

### ESCOPO:

Audit Trail, structured logs, correlation chains, system metrics, AI metrics e trading metrics.

### FORA DO ESCOPO:

Alteração de regras de negócio.

#### DEPENDÊNCIAS:

Fases 01-13.

#### VISÃO ARQUITETURAL:

Toda operação deve ser reconstruível através da cadeia:

MarketState

->

AI Run

->

TradeIntent

->

RiskDecision

->

Order

->

Fill

->

Position

->

Portfolio

#### CRITÉRIOS DE ACEITE:

- Toda operação possui correlation\_id.
- Audit events são persistidos.
- Logs são estruturados.
- Métricas técnicas existem.
- Métricas de IA existem.
- Métricas de trading existem.
- Uma operação pode ser reconstruída integralmente.

#### TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.

Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.

6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

**IMPORTANTE:**

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprove por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

**STATUS:**

PASS / FAIL / ARCHITECTURAL\_BLOCKER

**SUMMARY:**

resumo objetivo

**IMPLEMENTED:**

o que foi encontrado implementado

**MISSING:**

o que deveria existir e não existe

**OUT\_OF\_SCOPE:**

funcionalidades indevidas

**ARCHITECTURAL\_VIOLATIONS:**

violações encontradas

**TEST\_RESULTS:**

testes executados e resultado

**SECURITY\_RESULTS:**

resultado

**EVIDENCE:**

arquivos, testes e evidências

**RECOMMENDATION:**

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 14 – AUDIT & OBSERVABILITY

=====

**OBJETIVO ESPERADO:**

Consolidar rastreabilidade completa e métricas.

**ESCOPO ESPERADO:**

Audit Trail, structured logs, correlation chains, system metrics, AI metrics e trading metrics.

**FORA DO ESCOPO:**

Alteração de regras de negócio.



#### DEPENDÊNCIAS:

Fases 01-13.

#### VISÃO ARQUITETURAL:

Toda operação deve ser reconstruível através da cadeia:

```
MarketState
->
AI Run
->
TradeIntent
->
RiskDecision
->
Order
->
Fill
->
Position
->
Portfolio
```

#### CRITÉRIOS DE ACEITE:

- Toda operação possui correlation\_id.
- Audit events são persistidos.
- Logs são estruturados.
- Métricas técnicas existem.
- Métricas de IA existem.
- Métricas de trading existem.
- Uma operação pode ser reconstruída integralmente.

#### PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 15 — Resilience, Security & Chaos

## Objetivo

Validar que falhas não provocam comportamento inseguro.

## Escopo

Chaos testing, restart testing, provider failures, database failures, Redis failure, worker failure, websocket failure, market gaps, duplicate events, duplicate orders, broker unavailable, security checks e backup/restore.

## Fora do Escopo

Novas funcionalidades.

## Dependências

Fases 01-14.

## Visão Arquitetural

O sistema deve preferir PAUSE/SAFE STATE quando não consegue determinar corretamente seu estado.

## CrITÉrios de Aceite

- PostgreSQL down não gera novas ordens.
- Redis down não corrompe source of truth.
- Worker crash permite recovery.
- LLM timeout bloqueia nova entrada.
- Market gap bloqueia decisão quando necessário.
- Duplicate event não duplica operação.
- Restart durante execução é reconciliado.
- Backup/restore funciona.
- Secrets permanecem protegidos.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.
6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.

8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 15 – RESILIENCE, SECURITY & CHAOS

=====

OBJETIVO:

Validar que falhas não provocam comportamento inseguro.

ESCOPO:

Chaos testing, restart testing, provider failures, database failures, Redis failure, worker failure, websocket failure, market gaps, duplicate events, duplicate orders, broker unavailable, security checks e backup/restore.

FORA DO ESCOPO:

Novas funcionalidades.

DEPENDÊNCIAS:

Fases 01-14.

VISÃO ARQUITETURAL:

O sistema deve preferir PAUSE/SAFE STATE quando não consegue determinar corretamente seu estado.

CRITÉRIOS DE ACEITE:

- PostgreSQL down não gera novas ordens.
- Redis down não corrompe source of truth.
- Worker crash permite recovery.

- LLM timeout bloqueia nova entrada.
- Market gap bloqueia decisão quando necessário.
- Duplicate event não duplica operação.
- Restart durante execução é reconciliado.
- Backup/restore funciona.
- Secrets permanecem protegidos.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.  
Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 15 – RESILIENCE, SECURITY & CHAOS

=====

OBJETIVO ESPERADO:

Validar que falhas não provocam comportamento inseguro.

ESCOPO ESPERADO:

Chaos testing, restart testing, provider failures, database failures, Redis failure, worker failure, websocket failure, market gaps, duplicate events, duplicate orders, broker unavailable, security checks e backup/restore.

FORA DO ESCOPO:

Novas funcionalidades.

DEPENDÊNCIAS:

Fases 01-14.

VISÃO ARQUITETURAL:

O sistema deve preferir PAUSE/SAFE STATE quando não consegue determinar corretamente seu estado.

CRITÉRIOS DE ACEITE:

- PostgreSQL down não gera novas ordens.
- Redis down não corrompe source of truth.
- Worker crash permite recovery.
- LLM timeout bloqueia nova entrada.
- Market gap bloqueia decisão quando necessário.
- Duplicate event não duplica operação.
- Restart durante execução é reconciliado.
- Backup/restore funciona.
- Secrets permanecem protegidos.

PROCEDIMENTO:

1. Inspeção os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.

3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FASE 16 — Final V1 Certification

## Objetivo

Certificar a V1 como PAPER\_READY.

## Escopo

Unit tests, integration tests, E2E, replay, backtest, security, chaos, recovery, backup/restore e paper trading.

## Fora do Escopo

Live trading.

## Dependências

Fases 01-15 aprovadas.

## Visão Arquitetural

Esta fase não deve introduzir arquitetura nova.

Ela somente verifica se o sistema implementado corresponde ao Blueprint V1.1.

## Critérios de Aceite

- Todas as fases anteriores aprovadas.
- Testes completos passam.
- Chaos testing passa.
- Security checks passam.
- Recovery passa.
- Backup/restore passa.
- Paper trading funciona.
- Audit trail é íntegro.
- Nenhum LiveBroker existe.
- V1\_STATUS = PAPER\_READY.

## Prompt de Implementação

Você está implementando uma fase do projeto Autonomous Swing Trading Bot.

A autoridade arquitetural é exclusivamente:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Você NÃO é o arquiteto.

Você deve implementar exatamente o escopo da fase solicitada.

REGRAS OBRIGATÓRIAS:

1. Não altere a arquitetura.
2. Não crie componentes arquiteturais não especificados.
3. Não altere Domain Contracts sem autorização formal.
4. Não altere State Machines sem autorização formal.
5. Não altere hard safety limits.

6. Não implemente LiveBroker.
7. Não permita caminho direto LLM -> Broker.
8. Não permita caminho Frontend -> Broker.
9. Não permita bypass do Risk Engine.
10. Não utilize float em cálculos financeiros críticos.
11. Utilize UTC como timezone interno.
12. Preserve correlation\_id.
13. Preserve idempotency.
14. Não introduza look-ahead.
15. Não armazene secrets em código ou logs.
16. Não exponha secrets ao frontend.
17. Não trate Redis como source of truth financeiro.
18. Preserve fail-closed.
19. Preserve recovery seguro.
20. Se algo não estiver especificado, NÃO invente.

Se encontrar uma decisão arquitetural não especificada:

marque como ARCHITECTURAL\_BLOCKER

e não tome a decisão por conta própria.

ANTES DE CODAR:

- leia a estrutura existente;
- leia o Blueprint;
- leia ADRs relevantes;
- verifique dependências das fases anteriores;
- identifique contratos existentes;
- não reimplente componentes já existentes.

DURANTE A IMPLEMENTAÇÃO:

- mantenha o escopo estritamente limitado à fase;
- escreva testes;
- não faça refatorações arquiteturais fora do escopo;
- mantenha compatibilidade com as fases anteriores.

AO FINAL:

- execute os testes;
- reporte exatamente os arquivos modificados;
- reporte decisões tomadas;
- reporte qualquer ARCHITECTURAL\_BLOCKER;
- forneça evidências de aceite;
- não declare a fase concluída se algum critério falhar.

=====

FASE 16 – FINAL V1 CERTIFICATION

=====

OBJETIVO:

Certificar a V1 como PAPER\_READY.

ESCOPO:

Unit tests, integration tests, E2E, replay, backtest,  
security, chaos, recovery, backup/restore e paper trading.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 01-15 aprovadas.

VISÃO ARQUITETURAL:

Esta fase não deve introduzir arquitetura nova.

Ela somente verifica se o sistema implementado corresponde  
ao Blueprint V1.1.

CRITÉRIOS DE ACEITE:

- Todas as fases anteriores aprovadas.



- Testes completos passam.
- Chaos testing passa.
- Security checks passam.
- Recovery passa.
- Backup/restore passa.
- Paper trading funciona.
- Audit trail é íntegro.
- Nenhum LiveBroker existe.
- V1\_STATUS = PAPER\_READY.

TAREFA:

Implemente exclusivamente esta fase.

Não apenas crie estruturas vazias para aparentar conclusão.  
A implementação deve ser funcional dentro do escopo definido.

Crie testes suficientes para demonstrar os critérios de aceite.

Ao finalizar, forneça:

1. Resumo da implementação.
2. Lista de arquivos criados/modificados.
3. Testes executados.
4. Resultado dos testes.
5. Critérios de aceite e evidência correspondente.
6. ARCHITECTURAL\_BLOCKERS, se houver.
7. Qualquer desvio encontrado.

Não declare APPROVED.  
Apenas informe que a implementação está pronta para auditoria.

## Prompt de Auditoria

Você é o auditor da fase atual do Autonomous Swing Trading Bot.

A autoridade arquitetural é:

BLUEPRINT V1.1

Status:

ARCHITECTURE FROZEN

Sua função NÃO é corrigir o código.

Sua função é AUDITAR.

Você deve verificar se a implementação corresponde exatamente ao escopo da fase e se não viola a arquitetura.

REGRAS:

1. Não aceite implementação apenas porque compila.
2. Leia o código efetivamente implementado.
3. Execute os testes relevantes.
4. Verifique os contratos.
5. Verifique as dependências.
6. Procure funcionalidades fora do escopo.
7. Procure bypasses arquiteturais.
8. Procure secrets expostos.
9. Procure uso de float em cálculos financeiros.
10. Procure problemas de timezone.
11. Procure look-ahead.
12. Procure ausência de idempotência onde aplicável.
13. Procure ausência de correlation\_id onde aplicável.
14. Procure estados inválidos.
15. Procure comportamento fail-open.
16. Verifique que LiveBroker NÃO foi criado.
17. Verifique que LLM NÃO possui acesso direto ao broker.
18. Verifique que frontend NÃO possui acesso direto ao broker.
19. Verifique que Risk Engine não pode ser bypassado.
20. Verifique se a implementação alterou algo fora do escopo.

IMPORTANTE:

Se encontrar qualquer violação arquitetural crítica:

STATUS = FAIL

Não aprobe por conveniência.

Se houver dúvida sobre um requisito não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Formato obrigatório do resultado:

STATUS:

PASS / FAIL / ARCHITECTURAL\_BLOCKER

SUMMARY:

resumo objetivo

IMPLEMENTED:

o que foi encontrado implementado

MISSING:

o que deveria existir e não existe

OUT\_OF\_SCOPE:

funcionalidades indevidas

ARCHITECTURAL\_VIOLATIONS:

violações encontradas

TEST\_RESULTS:

testes executados e resultado

SECURITY\_RESULTS:

resultado

EVIDENCE:

arquivos, testes e evidências

RECOMMENDATION:

APPROVE / FIX\_REQUIRED / ARCHITECTURAL\_REVIEW

=====

AUDITORIA DA FASE 16 – FINAL V1 CERTIFICATION

=====

OBJETIVO ESPERADO:

Certificar a V1 como PAPER\_READY.

ESCOPO ESPERADO:

Unit tests, integration tests, E2E, replay, backtest,  
security, chaos, recovery, backup/restore e paper trading.

FORA DO ESCOPO:

Live trading.

DEPENDÊNCIAS:

Fases 01-15 aprovadas.

VISÃO ARQUITETURAL:

Esta fase não deve introduzir arquitetura nova.

Ela somente verifica se o sistema implementado corresponde  
ao Blueprint V1.1.

CRITÉRIOS DE ACEITE:

- Todas as fases anteriores aprovadas.
- Testes completos passam.
- Chaos testing passa.
- Security checks passam.
- Recovery passa.
- Backup/restore passa.
- Paper trading funciona.
- Audit trail é íntegro.
- Nenhum LiveBroker existe.
- V1\_STATUS = PAPER\_READY.

PROCEDIMENTO:

1. Inspecione os arquivos modificados.
2. Compare a implementação com o Blueprint V1.1.
3. Execute os testes.
4. Verifique os critérios de aceite individualmente.
5. Procure implementação fora do escopo.
6. Procure violações arquiteturais.
7. Procure riscos de segurança.
8. Procure regressões nas fases anteriores.
9. Verifique idempotência quando aplicável.
10. Verifique correlation\_id quando aplicável.
11. Verifique UTC quando aplicável.
12. Verifique Decimal quando aplicável.
13. Verifique fail-closed quando aplicável.
14. Verifique que nenhum LiveBroker foi criado.
15. Verifique que a IA não possui autoridade direta de execução.

Não corrija automaticamente.

Se houver problema:

STATUS = FAIL

Se houver requisito arquitetural não especificado:

STATUS = ARCHITECTURAL\_BLOCKER

Somente produza:

STATUS = PASS

quando todos os critérios estiverem demonstravelmente atendidos.

Gere o relatório no formato definido pelo prompt de auditoria.

## Gate

APPROVED somente quando implementação, testes, critérios de aceite e auditoria estiverem completos, sem ARCHITECTURAL\_BLOCKER pendente.

# FINAL V1 CERTIFICATION

A certificação final não introduz nova arquitetura. Ela verifica se a implementação corresponde ao Blueprint V1.1.

```
ALL PHASES PASS
  |
  v
FULL E2E
  |
  v
REPLAY / BACKTEST
  |
  v
SECURITY AUDIT
  |
  v
CHAOS TESTING
  |
  v
RECOVERY TEST
  |
  v
BACKUP / RESTORE
  |
  v
PAPER TRADING
  |
  v
FINAL APPROVAL
  |
  v
V1_STATUS = PAPER_READY
```

## Architecture Freeze

O Blueprint V1.1 é a autoridade arquitetural do projeto.

Nenhum agente ou desenvolvedor pode modificar arquitetura, contratos, State Machines, limites de risco ou escopo sem alteração formal de versão e ADR.

Requisito não especificado deve gerar ARCHITECTURAL\_BLOCKER.

## Estado Final da V1

```
V1_STATUS = PAPER_READY

LIVE_READY = FALSE
LIVE_BROKER = NOT_IMPLEMENTED
```